

Gametracker

Estructura y diseño de la web

Durante el desarrollo de un proyecto es de suma importancia establecer las bases para su creación, a continuación, te comparto la lógica estructural desde mi punto de vista para tu proyecto:

Backend

A. Revisa y analiza la documentación inicial que te proporcionan:

<https://github.com/JacoboGarcesO/reto-final> en esta documentación te proporcionan dos (esto cambia dependiendo de cómo veas tu proyecto dado que puedes estructurarlo de diferente manera, en este caso tenemos a la entidad videojuegos:

```
{
  _id: ObjectId,
  titulo: String,
  genero: String,    // "Acción", "RPG", "Estrategia", etc.
  plataforma: String, // "PC", "PlayStation", "Xbox", etc.
  añoLanzamiento: Number,
  desarrollador: String,
  imagenPortada: String, // URL de la imagen
  descripcion: String,
  completado: Boolean,
  fechaCreacion: Date
}
```

Y por el otro lado tienes la entidad reseñas (review):

```
{
  _id: ObjectId,
  juegoId: ObjectId, // Referencia al videojuego
  puntuacion: Number, // 1-5 estrellas
  textoReseña: String,
  horasJugadas: Number,
  dificultad: String, // "Fácil", "Normal", "Difícil"
```

```
recomendaria: Boolean,  
fechaCreacion: Date,  
fechaActualizacion: Date  
}
```

B. Endpoints:

Tienes una gran ventaja dado que la misma documentación te indican los endpoints para cada entidad en este caso sabemos que poseemos de un CRUD (Create, Read, Update and Delete) o en la práctica llamados como GET (obtiene), GETBYID (obtener por id), POST (crear), PUT (en este caso tu PUT será por id) y el Delete que también se maneja por id, en el caso de la entidad Videojuegos su Controller tendrá estos métodos:

- GET /api/juegos - Obtener todos los juegos de tu biblioteca
- GET /api/juegos/:id - Obtener un juego específico
- POST /api/juegos - Agregar juego a tu colección
- PUT /api/juegos/:id - Actualizar información del juego
- DELETE /api/juegos/:id - Remover juego de tu biblioteca

Y en el caso de la entidad reseñas sus principales métodos serán los siguientes:

- GET /api/juegos - Obtener todos los juegos de tu biblioteca
- GET /api/juegos/:id - Obtener un juego específico
- POST /api/juegos - Agregar juego a tu colección
- PUT /api/juegos/:id - Actualizar información del juego
- DELETE /api/juegos/:id - Remover juego de tu biblioteca

Guía de flujo de trabajo para el Backend (GameTracker)

Estructura de carpetas final

Antes de empezar, visualiza la estructura que vamos a crear. Esto te ayudará a entender dónde va cada pieza del rompecabezas:

```
/gametracker-backend
|-- /config
| |-- db.js      // Lógica de conexión a la base de datos
|-- /models
| |-- Game.js    // Schema y Modelo de Videojuegos
| |-- Review.js  // Schema y Modelo de Reseñas
|-- /routes
| |-- games.js   // Rutas para la API de juegos (/api/juegos)
| |-- reviews.js // Rutas para la API de reseñas (/api/reseñas)
|-- /controllers
| |-- gameController.js // Lógica de negocio para los juegos
| |-- reviewController.js // Lógica de negocio para las reseñas
|-- .env          // Variables de entorno (¡secreto!)
|-- .gitignore    // Archivos a ignorar por Git
|-- server.js     // Punto de entrada principal de la aplicación
|-- package.json
```

Fase 1: Configuración inicial

Crear el Proyecto:

- Crea una carpeta para tu backend (ej. gametracker-backend).
- Abre una terminal en esa carpeta y ejecuta `npm init -y`. Esto crea tu archivo `package.json`, que es el manifiesto de tu proyecto.

Instalar Dependencias:

- Instala las librerías esenciales que necesitarás.
- **Express:** El framework para construir el servidor.
- **Mongoose:** Para modelar y comunicarte con tu base de datos MongoDB.
- **dotenv:** Para manejar variables de entorno (como la URL de tu base de datos) de forma segura.
- **cors:** Para permitir que tu frontend (que estará en un dominio diferente durante el desarrollo) pueda comunicarse con tu backend.

npm install express mongoose dotenv cors

Te recomiendo instalar nodemon como una dependencia de desarrollo. Reiniciará automáticamente tu servidor cada vez que guardes un archivo:

npm install --save-dev nodemon

En tu package.json, añade un script para correr el servidor fácilmente:

```
"scripts": {  
  "start": "node server.js",  
  "dev": "nodemon server.js"  
}
```

Fase 2: conexión a la base de datos

Configurar MongoDB Atlas:

- Ve a [MongoDB Atlas](#), crea una cuenta gratuita y un nuevo clúster.
- Sigue los pasos para crear un usuario de base de datos y permitir el acceso desde cualquier dirección IP (0.0.0.0/0) para el desarrollo.
- Obtén tu URL de conexión.

Conectar la aplicación:

- Crea un archivo llamado “.env” en la raíz de tu proyecto. Guarda tu URL de conexión para mantenerla segura y fuera de tu código fuente.

MONGO_URI=mongodb+srv://<tu_usuario>:<tu_password>@...

Crea la carpeta “config” y dentro, el archivo db.js. Aquí pondrás la lógica para conectar con Mongoose.

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log('MongoDB Conectada Exitosamente ✅');
  } catch (error) {
    console.error(`Error: ${error.message}`);
    process.exit(1); // Detiene la app si no se puede conectar
  }
};

module.exports = connectDB;
```

Fase 3: definir los modelos de datos

Crear schemas:

- Crea la carpeta models.
- Dentro, crea Game.js y Review.js.
- Define el schema para cada uno, siguiendo el contexto que tienes.

```
const mongoose = require('mongoose');

const gameSchema = new mongoose.Schema({
  titulo: { type: String, required: true, trim: true },
  genero: { type: String, required: true },
  plataforma: { type: String, required: true },
  añoLanzamiento: { type: Number },
  desarrollador: { type: String },
  imagenPortada: { type: String }, // URL de la imagen
  descripcion: { type: String },
  completado: { type: Boolean, default: false }
}, {
  timestamps: true // Añade fechaCreacion y fechaActualizacion
  automáticamente
});

const Game = mongoose.model('Game', gameSchema);
module.exports = Game;
```

Fase 4: implementar la lógica (Controllers)

Crear controladores:

- Crea la carpeta controllers.
- Dentro, crea gameController.js.
- Define una función para cada endpoint de los juegos. Usa async/await para manejar las operaciones de la base de datos.

```
// controllers/gameController.js
const Game = require('../models/Game'); // Importa el modelo

// @desc    Obtener todos los juegos
// @route    GET /api/juegos
const getAllGames = async (req, res) => {
  try {
    const games = await Game.find({});
    res.status(200).json(games);
  } catch (error) {
    res.status(500).json({ message: 'Error al obtener los juegos' });
  }
};

// @desc    Agregar un nuevo juego
// @route    POST /api/juegos
const createGame = async (req, res) => {
  try {
    const { titulo, genero, plataforma } = req.body; // Extrae datos
    // del cuerpo de la petición
    const newGame = new Game({ titulo, genero, plataforma });
    const savedGame = await newGame.save();
    res.status(201).json(savedGame);
  } catch (error) {
    res.status(400).json({ message: 'Error al crear el juego' });
  }
};

// Exporta las funciones
module.exports = {
  getAllGames,
  createGame,
  // ... aquí irían getGameById, updateGame, deleteGame
};
```

Fase 5: Definir las rutas

Crear archivos de rutas:

- Crea la carpeta routes.
- Dentro, crea games.js.
- Define las rutas usando el Router de Express y asócialas con las funciones del controlador.

```
// routes/games.js
const express = require('express');
const router = express.Router();
const { getAllGames, createGame } =
require('../controllers/gameController');

// Ruta para obtener todos los juegos y crear un juego nuevo
router.route('/').get(getAllGames).post(createGame);

// Aquí añadirías las rutas para /:id
//
router.route('/:id').get(getGameById).put(updateGame).delete(deleteGame
);

module.exports = router;
```

Fase 6: ensamblar y encender el servidor

Configurar server.js:

- Importa todo lo que necesitas, conecta la base de datos y dile a Express que use tus rutas.

```
// server.js
const express = require('express');
const dotenv = require('dotenv');
const cors = require('cors');
const connectDB = require('./config/db');

// Cargar variables de entorno
dotenv.config();

// Conectar a la base de datos
connectDB();
```

```
const app = express();

// Middlewares
app.use(cors()); // Habilita CORS
app.use(express.json()); // Permite al servidor aceptar y parsear JSON

// Importar Rutas
const gameRoutes = require('./routes/games');

// Usar Rutas
app.use('/api/juegos', gameRoutes);
// app.use('/api/reseñas', reviewRoutes); // Cuando las crees

const PORT = process.env.PORT || 5000;

app.listen(PORT, () => console.log(`Servidor corriendo en el puerto ${PORT}`));
```

Fase 7: probar tu API

Inicia tu servidor: npm run dev.

Prueba los endpoints:

- Haz una petición GET a <http://localhost:5000/api/juegos> para obtener todos los juegos (al principio devolverá un array vacío []).
- Haz una petición POST a <http://localhost:5000/api/juegos> con un cuerpo JSON como { "titulo": "The Witcher 3", "genero": "RPG", "plataforma": "PC" } para crear tu primer juego.

Para hacer pruebas te recomiendo usar Postman o Insomnia ambos tienen su versión de escritorio y son fáciles de usar.

Frontend:

Inspirarte de las grandes empresas como Epic Games, Steam, itch.io, etc (hacer un benchmarking si es posible) y aplica en tu proyecto todo lo bueno que tienen cada una de ellas. También te aconsejo utilizar herramientas para modelar tu diseño como Figma en el siguiente enlace puedes unirte como estudiante o educador: <https://www.figma.com/education/>

esto te servirá para llenar tu documentación con mayor profesionalismo añadiendo tus casos de uso, mockups (UI), hasta la traza de la experiencia del usuario final (UX). También puedes apoyarte de Canva que posee un apartado de estudiantes y educadores, para registrarte puedes hacerlo desde el siguiente enlace: <https://www.canva.com/education/>

Una vez que tengas generado tu mockup el cual deseas implementar te aconsejo hacerlo con Vite dado que tiene como objetivo proporcionar una experiencia de desarrollo más rápida y ágil para proyectos web modernos para iniciar un proyecto puedes hacerlo de la siguiente forma:

- `npm create vite@latest`, si no usas npm tienes estas alternativas `yarn create vite`, `pnpm create vite`, `bun create vite`, `deno init --npm vite`

En el siguiente enlace encontrarás más sobre Vite <https://es.vite.dev/guide/>, esta es una salida generando un proyecto con Vite:

```
Need to install the following packages:
create-vite@8.0.2
Ok to proceed? (y) y
```

```
> npx
> create-vite

|
◇ Project name:
  gametracker
|
◆ Select a framework:
  ○ Vanilla
  ○ Vue
  ● React
  ○ Preact
  ○ Lit
  ○ Svelte
  ○ Solid
  ○ Qwik
  ○ Angular
  ○ Marko
  ○ Others
```

Según el documento no indican nada sobre el lenguaje de programación que proporciona Rect, esto lo puede elegir con base a tu experiencia en código.

```
> npx
> create-vite

|
|
| Project name:
| gametracker
|
| Select a framework:
| React
|
| Select a variant:
| ● TypeScript
| ○ TypeScript + React Compiler
| ○ TypeScript + SWC
| ○ JavaScript
| ○ JavaScript + React Compiler
| ○ JavaScript + SWC
| ○ React Router v7 ↗
| ○ TanStack Router ↗
| ○ RedwoodSDK ↗
| ○ RSC ↗
```

```
> npx
> create-vite

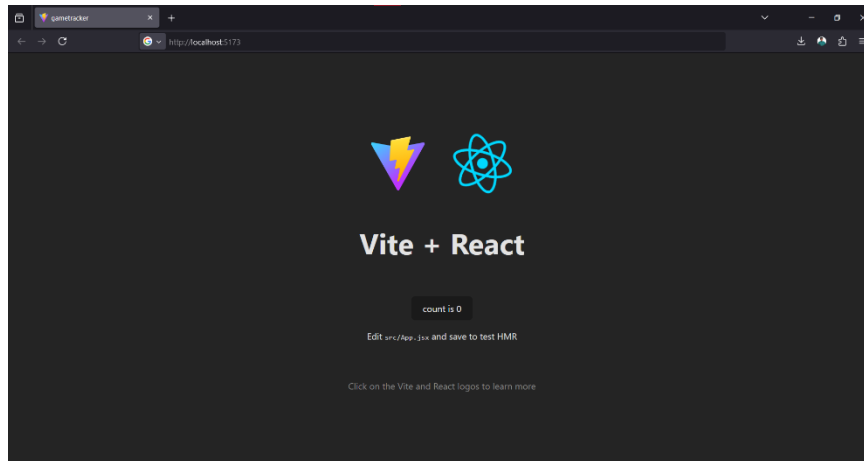
|
|
| Project name:
| gametracker
|
| Select a framework:
| React
|
| Select a variant:
| JavaScript
|
| Use rolldown-vite (Experimental)?:
| No
|
| Install with npm and start now?
| Yes
|
| Scaffolding project in C:\Users\SantosPedroBaltazarJ\Documents\FishCorp\Asesorias\gametracker...
|
| Installing dependencies with npm...
||
```

Una vez terminada la instalación de las dependencias obtendrás el enlace de tu proyecto ejecutándose de manera local (ver la siguiente página).

```
VITE v7.1.9 ready in 1054 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

Al abrir el enlace podrás observar tu aplicación con un pequeño proyecto de prueba



Para empezar a desarrollar en tu proyecto, terminar de ejecutar tu proyecto actual que estas visualizando en consola con `ctrl + c` y posteriormente escribir `cd gametracker` una vez dentro de la ruta escribe `code .` par abrir tu proyecto en Visual Studio Code y empezar a desarrollar. Recuerda que tu proyecto dependiendo del lenguaje que elijas tendrá una extensión diferente de archivo en mi caso es `.jsx` (para los componentes claro, luego tú podrás estructurar tu carpeta como desees dado que toma este video de midudev para guiarte https://youtube.com/shorts/4DP8mMUjWA8?si=b_NkUcj7AD3ejcL.

Además de ello te comparto unos puntos clave para tener un desarrollo eficaz implementa librerías de iniciación como React Router Dom y verifica si puedes usar Tailwindcss ya que te permitirá tener un diseño más reponsive (adaptativo), es importante recalcar que Tailwindcss trabaja de la resolución más pequeña a la más grande, además de que puedes personalizar estos tamaños con sus respectivas etiquetas como `sm`, `md`, `lg`, `xl`, `2xl`, etc; un paso importante es realización de pruebas y

estas las puedes hacer <https://vitest.dev/> además te comparto una lista de herramientas categorizadas:

Para pruebas E2E (End-to-End)

Cypress → Muy popular, rápido y fácil de usar.

Página: <https://www.cypress.io>

Playwright (Microsoft) → Más moderno, compatible con múltiples navegadores.

Página: <https://playwright.dev>

Selenium → Más antiguo, multiplataforma y compatible con muchos lenguajes.

Para pruebas unitarias

Jest (para proyectos en JavaScript/React/Vue)

Mocha o **Vitest** (más ligero, ideal para Vite)

Para pruebas visuales

Percy

Applitools

Lokalise Visual Context (para pruebas con localización)

Si requieren de más apoyo pueden escribirme un mensaje por este medio o por correo a santospedrobaltazarjctowork@gmail.com, estaré al tanto para brindar apoyo en cuanto me sea posible. Saludos Santos.